

어셈블러의 동작과정을 본다

핵 어셈블러를 개발할 것이다.

이 프로젝트는 6장 이후로 나오는 7개의 소프트웨어 개발 프로젝트중 첫번째이다.

번역기(어셈블러, 가상머신, 컴파일러) 소프트웨어 프로젝트들은 다른 프로그래밍 언어를 이용해서 구현해도 된다

1.

기계어는 보통 기호형과 2진 형식으로 정의 된다.

어셈블리에서는 일반적으로 2가지 방법으로 기호를 도입한다.

-1. 변수 : 변수명으로 기호를 사용하면 번역기는 자동적으로 그 기호를 주소에 할당한다.

-2. 레이블 : 프로그램 내 다양한 위치를 기호로 표시할 수 있다.

(loop) 이런 것들

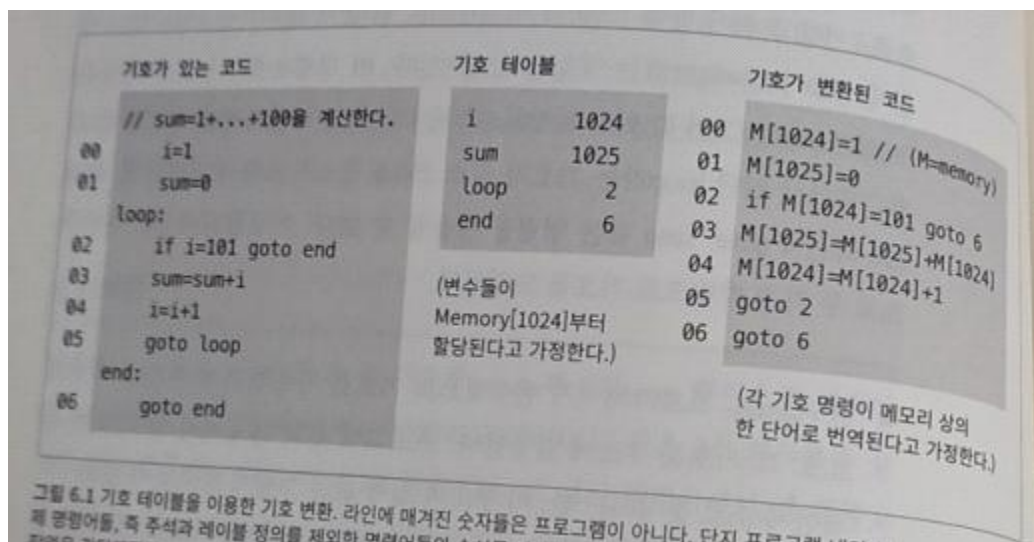
미리 정의된 기호를 역시 미리 정의된 2진 코드로 바꾸는 작업은 복잡하지 않다.

다만, 사용자 정의 변수명과 기호 레이블을 실제 메모리 주소에 매핑하는 일이 어렵다.

하드웨어에서 소프트웨어로 전환되는 과정 중 가장 복잡한 작업이 "기호변환작업"이다.

*기호 변환

2개의 변수와 2개의 레이블



기호없는 코드로 바꾸려면 어떻게 해야하는가?

임의로 규칙을 만들어보자.

1.번역된 코드는 주소0부터 시작하는 메모리에 저장

2.변수들은 주소 1024부터 할당한다.

3.소스코드에 새로운 기호xxx가 나타날 때마다 (xxx,n) 라인을 테이블에 추가하는 방식으로 기호 테이블에 구성한다. (n은 기호와 연결된 메모리 주소)

4. 기호 테이블이 다 만들어지면 프로그램을 기호가 없는 코드로 옮긴다.

번역기는 프로그램 소스에 있는 명령들이 각각 생성하는 단어가 몇 개인지 추적하고, 그에 따라 명령어 메모리 계수기에 업데이트 한다.

*어셈블러

어셈블러는 어셈블리 명령 열을 입력받고, 동일한 의미의 2진 명령어 열로 출력한다.
결과 코드는 컴퓨터 메모리에 로드되어 실행된다. (번역된 2진 명령어 코드가 저장된다)

어셈블러는 번역 기능이 있는 텍스트 처리 프로그램이다.
어셈블러를 작성할 때는 1)어셈블리 문법에 대한 전체 문서
2)그에 대응하는 2진 코드 목록이 있어야한다.
보통 “기계어 명세”라 불리는 규칙 따르면 어렵지 않다.

- *구문분석 >> 기호 명령내 필드들을 식별
- *각 필드에 대응하는 기계어 비트 생성
- *모든 기호 참조를 메모리 주소를 가리키는 숫자로 바꾼다
- *2진 코드들을 조립하여 완전한 기계 명령어로 바꾼다.
- (마지막 기호 처리 작업은 어렵다>)

2. 핵 어셈블리-2진 번역 명세서

- 1)2진 기계어 코드는 ‘hack’ , 어셈블리 코드 프로그램은 ‘asm’ 확장자 쓴다.
asm 번역하면 hack파일 생성
- 2)2진 코드 파일은 텍스트 라인으로 이루어진다.
각 라인마다 0과 1로 이루어진 ASCII 문자열이다.
기계어 프로그램을 불러올 때 n번째 라인의 2진 코드는 명령어 메모리의 주소n에 저장.
- 3)asm파일
명령어(A-명령어,C-명령어)나 기호선언 라인들로 구성.
- 4)상수와 기호
변수 네이밍 규칙(굳이 정리 안하겠다)
- 5)주석
- 6)공백은 무시한다. 빈 라인도 무시한다.
- 7)대소문자 규칙: 어셈블리 연상기호는 전부 대문자로 쓴다.
사용자 정의 레이블/변수명은 대소문자 구분을 한다.

3.명령어

명령어는 C/A명령어 2가지 있다.

4. 기호

핵 어셈블리 명령은 상수나 기호를 이용해 메모리 주소를 참조할 수 있다.

선언 기호 핵 어셈블리 프로그램에서는 다음과 같은 선언 기호 predefined symbol
를 사용할 수 있다.

레이블	RAM 주소	(hexa)
	0	0x0000
SP	1	0x0001
LCL	2	0x0002
ARG	3	0x0003
THIS	4	0x0004
THAT		
R0-R15	0-15	0x0000-f
SCREEN	16384	0x4000
KBD	24576	0x6000

5. 레이블 기호

의사명령(xxx)는 기호 xxx가 프로그램의 다음 명령이 있는 명령어 메모리 위치를 참조하도록 정의한다.

한 레이블은 한 번만 정의할 수 있다.

6. 변수 기호

변수는 등장한 순서대로 RAM 주소 16에서 시작하는 메모리 주소에 차례대로 매핑된다.
ex)

어셈블리 코드 (Prog.asm)	2진 코드 (Prog.hack)
<pre>// Adds 1 * ... + 100 @i M=1 // i=1 @sum M=0 // sum=0 (LLOOP) @i D=M // D=i @100 D=D-A // D=i-100 @END D;JGT // if (i-100)>0 goto END @i D=M // D=i @sum M=D+M // sum=sum+i @i M=M+1 // i=i+1 @LOOP @;JMP // goto LOOP (END) @END @;JMP // 무한 루프</pre>	<pre>(생략) 0000 0000 0001 0000 1110 1111 1100 1000 0000 0000 0001 0001 1110 1010 1000 1000 (생략) 0000 0000 0001 0000 1111 1100 0001 0000 0000 0000 0110 0100 1110 0100 1101 0000 0000 0000 0001 0010 1110 0011 0000 0001 0000 0000 0001 0000 1111 1100 0001 0000 0000 0000 0001 0001 1111 0000 1000 1000 0000 0000 0001 0000 1111 1101 1100 1000 0000 0000 0000 0100 1110 1010 1000 0111 (생략) 0000 0000 0001 0010 1110 1010 1000 0111</pre>

어셈블러

동일한 프로그램의 어셈블리 코드와 2진 코드 버전

6. 구현

핵 어셈블러는 Prog.asm이라는 텍스트 파일을 입력 받아서

Prog.hack이라는 텍스트 파일을 출력한다.

입력 파일명은 어셈블러 프로그램의 커맨드 라인 인수로 입력한다

prompt> Assembler Prog.asm

어셈블리 명령에서 연상기호 부분(필드)은 대응되는 비트 코드로 옮겨진다.

code	d1	d2	d3	jump	j1	j2	j3
null	0	0	0	null	0	0	0
N	0	0	1	JGT	0	0	1
O	0	1	0	JEQ	0	1	0
PO	0	1	1	JGE	0	1	1
MO	1	0	0	JLT	1	0	0
A	1	0	1	JNE	1	0	1
AN	1	1	0	JLE	1	1	0
AD	1	1	1	JMP	1	1	1

기호는 명시된 메모리 주소로 변환된다.

레이블	RAM 주소	(hexa)
	0	0x0000
SP	1	0x0001
LCL	2	0x0002
ARG	3	0x0003
THIS	4	0x0004
THAT	0-15	0x0000-f
R0-R15	16384	0x4000
SCREEN	24576	0x6000
KBD		

*어셈블러는 4개의 모듈

-입력을 구문 분석하는 Parser 모듈

-모든 어셈블리 연상기호들의 2진 코드들을 제공하는 Code모듈(데이터 베이스)

-기호를 처리하는 SymbolTable

-전체 번역과정을 실행하는 메인 프로그램

이 4가지로 어셈블러는 구현한다.

*API 표기법

어셈블러/가상머신/컴파일러 챕터에서는 개발할 프로그래밍 언어를 다양하게 선택할 수 있다.

언어에 독립적인 API 기반으로 구현에 관한 가이드라인을 줄 것이다.

-보통의 프로젝트에서 API는 하나 이상의 루틴을 포함하는 몇개의 모듈 정의로 이루어진다.

-자바/c++/c#같은 객체 지향언어에서 모듈은 클래스, 루틴은 메서드에 대응한다.

-절차형 언어에서 루틴은 함수나 서브루틴, 프로시저에 대응하고,

모듈은 관련 데이터들을 처리하는 루틴들의 집합에 대응한다.

-

<실습을 위한 정리>

1. 프로그램을 위해 다루어야 하는 것

- 공백(무시)
- 명령어(a / c 명령어 구분)
- 심볼
 - (
 - 3가지 있다.
 - 사전정의 심볼
 - 레이블 심볼(레이블 다음에 오는 명령어의 실질적 ROM 주소를 담는다.)
 - 변수 심볼(생성되는 순서대로 16부터 1나씩 먹고 간다.
 -)

2. HackAssembler의 기능은 3가지로 나눈다

- 1). Parser : 읽고 명령어 파싱
- 2). Code : 2진코드 작성
- 3). SymbolTable : symbol을 다룬다.

3. HackAssembler의 플로우

아래 과정에서 Parser/Code/SymbolTable 의 기능을 잘 사용한다.

1). Initialize

- Prog.asm파일을 열고 변환작업 준비.
- Symbol table만들고 predefined symbol을 여기에 추가한다.

2). First Pass

1회차

- 프로그램 줄을 하나씩 읽고 오지 라벨 선언만을 집중해서 본다.
- 심볼 테이블을 작성한다.

3). Second Pass

2회차

- 이게 메인 루프이다.
- 명령어 받고 파싱한다.
- @명령어는 <symbol, value>로 테이블에 담는다. 그리고 2진 코드로 변환
- D명령어는 3개의 필드(dest, comp, jmp)를 각기 2진 코드로 변환한다
- 그리고 각 비트를 합쳐 string 타입의 16비트로 반환한다.

4. Parse API

*API

API(Application Programming Interface) 응용 프로그램 프로그래밍 인터페이스

응용 프로그램에서 사용할 수 있도록, 운영 체제나 프로그래밍 언어가 제공하는 기능을 제어할 수 있도록 만든 인터페이스이다.

주로 파일 제어, 창 제어, 화상 처리, 문자 제어등을 위한 인터페이스를 제공한다.

****어떤 플랫폼에서 기능을 사용할 수 있게 만든 인터페이스(일종의 모듈.함수)

*Parse

-문장을 문법적으로 분석하다.

-정해진 포맷으로 작성된 프로그램을 분석하여 실행할 수 있는 내부 코드로 변경하는 것.

객체지향언어에서 모듈은 클래스, 루틴은 메소드

1). 루틴

-Constructor/ initializer : Parser 생성하고 소스코드를 담은 텍스트 파일을 연다.

-현재 명령어 받기

-hasMoreLines() : 더 일할 것이 있는지 체크한다. boolean형 반환

-advance() : 다음 명령어 받는다. 현재 명령어 생성(string)

-현재 명령어 파싱

-instructionType() : 현재 명령어의 타입을 반환한다.

@xxx : A_INSTRUCTION

dest=comp;jmp : C_INSTRUCTION

(label) : L_INSTRUCTION

-symbol() : 명령어의 심볼 반환 (string 형으로 반환)

@sum : symbol() returns "sum"

-dest() : 명령어의 dest 필드 string형으로 반환

-comp() : 명령어의 comp 필드 string형으로 반환

-jump() : 명령어의 jmp 필드 string형으로 반환

D=D+1 : dest() "D" comp() "D+1" jump() "JLE"

M=-1 : dest() "M" comp() "-1" jump() null

5. Code API

2진 코드 생성부분

1). 루틴

- dest(string) : dest field 받아서 2진 코드로 변환
- comp(string) : comp field 받아서 2진 코드로 변환
- jump(string) : jump field 받아서 2진 코드로 변환

<i>comp</i>		<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>
0		1	0	1	0	1	0
1		1	1	1	1	1	1
-1		1	1	1	0	1	0
D		0	0	1	1	0	0
A	M	1	1	0	0	0	0
!D		0	0	1	1	0	1
!A	!M	1	1	0	0	0	1
-D		0	0	1	1	1	1
-A	-M	1	1	0	0	1	1
D+1		0	1	1	1	1	1
A+1	M+1	1	1	0	1	1	1
D-1		0	0	1	1	1	0
A-1	M-1	1	1	0	0	1	0
D+A	D+M	0	0	0	0	1	0
D-A	D-M	0	1	0	0	1	1
A-D	M-D	0	0	0	1	1	1
D&A	D&M	0	0	0	0	0	0
D!A	D!M	0	1	0	1	0	1

B == 0 *B == 1*

<i>dest</i>		<i>d</i>	<i>d</i>	<i>d</i>
null		0	0	0
M		0	0	1
D		0	1	0
DM		0	1	1
A		1	0	0
AM		1	0	1
AD		1	1	0
ADM		1	1	1

<i>jump</i>		<i>j</i>	<i>j</i>	<i>j</i>
null		0	0	0
JGT		0	0	1
JEQ		0	1	0
JGE		0	1	1
JLT		1	0	0
JNE		1	0	1
JLE		1	1	0
JMP		1	1	1

Examples:

dest("DM") returns "011"

comp("A+1") returns "0110111"

comp("D&M") returns "1000000"

jump("JNE") returns "101"

6. SymbolTable API

심볼을 다루는 부분

1).루틴

- Constructor/Initializer : SymbolTable 생성
- void addEntry(String symbol, int address) :<symbol,address>를 Table에 추가
- boolean contains(String symbol) : Table에 Symbol이 있는지 확인한다.
- int getAddress(String symbol) : symbol에 관련된 주소를 반환한다.

Symbol table: (example)	<i>symbol</i> <i>address</i>	
	<i>symbol</i>	<i>address</i>
	R0	0
	R1	1
	R2	2
	...	...
	R15	15
	SCREEN	16384
	KBD	24576
	SP	0
	LCL	1
	ARG	2
	THIS	3
	THAT	4
	LOOP	4
	STOP	18
	i	16
	sum	17

7. 클래스별 메소드 정리(모듈별 루틴 정리)

Parser module:

<i>Routine</i>	<i>Arguments</i>	<i>Returns</i>	<i>Function</i>
Constructor / initializer	Input file or stream	—	Opens the input file/stream and gets ready to parse it.
hasMoreLines	—	boolean	Are there more lines in the input?
advance	—	—	Skips over whitespace and comments, if necessary. Reads the next instruction from the input, and makes it the current instruction. This method should be called only if hasMoreLines is true. Initially there is no current instruction.
instructionType	—	A_INSTRUCTION, C_INSTRUCTION, L_INSTRUCTION (constants)	Returns the type of the current instruction: A_INSTRUCTION for @xxx, where xxx is either a decimal number or a symbol. C_INSTRUCTION for dest=comp;jump L_INSTRUCTION for (xxx), where xxx is a symbol.
symbol	—	string	If the current instruction is (xxx), returns the symbol xxx. If the current instruction is @xxx, returns the symbol or decimal xxx (as a string). Should be called only if instructionType is A_INSTRUCTION or L_INSTRUCTION.
dest	—	string	Returns the symbolic dest part of the current C-instruction (8 possibilities). Should be called only if instructionType is C_INSTRUCTION.
comp	—	string	Returns the symbolic comp part of the current C-instruction (28 possibilities). Should be called only if instructionType is C_INSTRUCTION.
jump	—	string	Returns the symbolic jump part of the current C-instruction (8 possibilities). Should be called only if instructionType is C_INSTRUCTION.

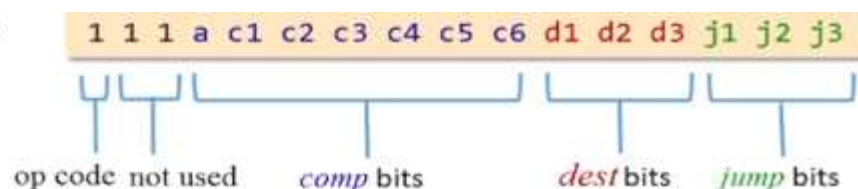
Code module:

<i>Routine</i>	<i>Arguments</i>	<i>Returns</i>	<i>Function</i>
dest	string	3 bits, as a string	Returns the binary code of the dest mnemonic.
comp	string	7 bits, as a string	Returns the binary code of the comp mnemonic.
jump	string	3 bits, as a string	Returns the binary code of the jump mnemonic.

SymbolTable module:

<i>Routine</i>	<i>Arguments</i>	<i>Returns</i>	<i>Function</i>
Constructor	—	—	Creates a new empty symbol table.
addEntry	symbol (string), address (int)	—	Adds <symbol, address> to the table.
contains	symbol (string)	boolean	Does the symbol table contain the given symbol?
getAddress	symbol (string)	int	Returns the address associated with the symbol.

Binary syntax:



8. 메인 플로우

각 클래스별 메소드 작성 후, 결정한다.

3.HackAssembler의 플로우

아래 과정에서 Parser/Code/SymbolTable 의 기능을 잘 사용한다.

<Initial>

- 1.Prog.asm파일 열기 >> Parser 생성자
- 2.Symbol table만들기 >> SymbolTable 생성자
- 3.Symbol table에 predefined symbol 추가 >> SymbolTable.addEntry()

<First Pass>

- 1.프로그램 줄을 하나씩 읽는다. >> Parser.advance()
- 2.레이블만 심볼 테이블에 작성한다. >> SymbolTable.addEntry()

<Second Pass>

file의 맨 처음부터 다시 시작한다.

- 1.명령어 받고 파싱 >> SymbolTable.addEntry() / Code.dest,comp,jump()
 - 파일 한 줄을 읽고 명령어만 구분한 후
 - @명령어는 <symbol, value>로 테이블에 담는다. 그리고 바로 2진 코드로 변환
 - D명령어는 3개의 필드(dest, comp, jmp)를 바로 2진 코드로 변환한다
- 2.이를 string으로 output file로 출력한다. >>파일 출력

Symbolic code

```
// Computes R1=1 + ... + R0
// 1 = 1
@1
M=1
// sum = 0
@sum
M=0
(LOOP)
// if i>R0 goto STOP
@i
D=M
@R0
D=D-M
@STOP
D;JGT
// sum += 1
@i
D=M
@sum
M=D+M
// i++
@i
M=M+1
@LOOP
0;JMP
(STOP)
@sum
D=M
...
```

Symbol table

symbol	value
R0	0
R1	1
R2	2
...	...
R15	15
SCREEN	16384
KBD	24576
SP	0
LCL	1
ARG	2
THIS	3
THAT	4
LOOP	4
STOP	18
i	16
sum	17

A data structure that the assembler creates and uses during the program translation

Initialization:

Creates the symbol table and adds the predefined symbols to the table

First pass: Counts lines and adds the label symbols to the table

Second pass:

- Generates binary code; in the process:
- Adds the variable symbols to the table

(details, soon)

<어셈블러에 대한 고찰>

가장 초기의 어셈블러는 하드웨어 형태일 것이다.

이진코드에서 문자로 바꾸는 과정이 필요한데, 문자는 인간 입장에서야 문자인 것이지만 RAM에 저장되는 것은 ASCII로 정의된 숫자이다.

RAM에 저장된 값을 읽고 판단하는 과정이 필요하므로 또 다른 하드웨어를 만들어서 RAM에 숫자로 저장된 어셈블리어 코드를 분석하는 것이다.

컴퓨터가 발전하면서 MCU, MPU 이런 것들이 발전하였고,

이런 모듈을 코딩하여 어셈블러를 만든다. 그러다 보니 어셈블러는 소프트웨어가 된 것이다.

하지만, MCU, MPU 내부 회로를 보면 결과적으로 하드웨어이므로

어셈블러는 하드웨어를 거쳐야하는 것이다.

<i>jump</i>	<i>j</i>	<i>j</i>	<i>j</i>	<i>effect:</i>
null	0	0	0	no jump
JGT	0	0	1	if <i>comp</i> > 0 jump
JEQ	0	1	0	if <i>comp</i> = 0 jump
JGE	0	1	1	if <i>comp</i> ≥ 0 jump
JLT	1	0	0	if <i>comp</i> < 0 jump
JNE	1	0	1	if <i>comp</i> ≠ 0 jump
JLE	1	1	0	if <i>comp</i> ≤ 0 jump
JMP	1	1	1	Unconditional jump